make it simpler we will go with simple YAML based declarations. We will write a separate article with a Helm chart sometime in future.

1. Create an EKS cluster in AWS cloud for the Tekton pipeline to run. Tekton pipelines will run inside a Kubernetes/OpenShift cluster; so this is a mandatory step.
2. Create a folder named IAC and create two sub-folders called AWS and GitOps.
3. The AWS folder will hold all infrastructure related workload creation tasks like S3 bucket, CloudWatch, EBS, etc.
4. The GitOps folder will consist of four sub-folders for different environments — *dev*, *test*, *stage* and *prod* folders. These folders will have the configuration definition as per their environment.
5. In the AWS folder, create cloudformation templates for creating security groups, load balancers, Apigateway, Route53 hosted zones, Alb controllers, etc.
6. Create a task for running the cloudformation template and name it as infratask.
7. Create a service account associated with roles and name spaces. Provide access tokens for Git, Docker Hub and EKS complete access.
8. Get access tokens for git access (read and write access), repository access and EKS deployment access with appropriate roles.
9. Create tasks for GITcheckout, build, package to Docker image and push it into image repository like Nexus or AWS ECR or Docker image repository. (You need to have tokens for this.)
10. Create a Tekton pipeline which consists of all the tasks sequentially or parallelly (as per your use case).
11. Create a pipeline run file which consists of the exact value of the git paths, storage volume, etc.
12. Docker containers are created using kaniko or buildah. These tasks are already available in Tekton Hub, which we can download and refer to in our pipeline.

Let us write a Tekton example code for pulling the Java code from the Git repository, build using Maven and package, create a Docker image, push it to Docker registry and deploy it into AWS EKS.

```
apiVersion: tekton.dev/v1beta1
kind: Pipeline
metadata:
  name: java-maven-docker-pipeline
spec:
  resources:
    - name: source-repo
      type: git
    - name: docker-image
      type: image
  tasks:
    - name: clone-repo
      taskRef:
        name: git-clone
      resources:
        inputs:
          - name: url
            resource: source-repo
          - name: revision
            resource: source-repo
      params:
        - name: path
          value: /workspace/source

    - name: build-with-maven
      taskRef:
        name: maven
      resources:
        inputs:
          - name: workspace
            resource: source-repo
      params:
        - name: PATH_TO_POM
          value: /workspace/source/path/
to/pom.xml
        - name: GOALS
          value: "clean package"

    - name: build-docker-image
      taskRef:
        name: buildah
      runAfter:
        - build-with-maven
      resources:
        inputs:
          - name: source
            resource: source-repo
      params:
        - name: DOCKERFILE
          value: /workspace/source/
Dockerfile
        - name: CONTEXT
          value: /workspace/source
        - name: IMAGE
          value: $(outputs.resources.
docker-image.url)

    - name: push-to-docker-registry
      taskRef:
        name: docker-push
      runAfter:
        - build-docker-image
      resources:
        inputs:
          - name: image
            resource: docker-image
      params:
        - name: TAG
          value: latest
        - name: IMAGE
          value: $(inputs.resources.
image.url)
        - name: REGISTRY
          value: DOCKER_REGISTRY
        - name: USERNAME
          value: your-docker-registry-
username
        - name: PASSWORD
          value: your-docker-registry-
password

    - name: deploy-to-eks
      taskRef:
        name: eks-deploy
      runAfter:
        - push-to-docker-registry
      resources:
        inputs:
          - name: source
            resource: source-repo
      params:
        - name: KUBECONFIG
          value: /workspace/.kube/config
        - name: CONTEXT
```